

Code Logic

```
#importing relevant libraries
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
```

Importing necessary python modules and libraries

SparkSession is the entry point to functions. Functions is used to get set of pre-defined functions and types for defining schema.

Implementing user defined functions:

1. Total Cost UDF

```
#creating user-defined functions
# 1. Calculate total cost
def calc_total_cost(items,t_type):
    total_cost=0
    for item in items:
        total_cost+=item['unit_price']*item['quantity']
    if t_type=='RETURN':
        return total_cost*-1
    else:
        return total_cost
```

To calculate the total revenue for each invoice, I multiplied the unit price of each product by its quantity to determine individual product revenues. I then summed these values to obtain the total invoice amount. For return transactions, I represented the total as a negative value to reflect the revenue reversal.

2. Total items UDF

```
# 2. Calculate total items
def calc_total_items(items):
    item_count=0
    for item in items:
        item_count+=item['quantity']
    return item_count
```

To determine the total number of products in each invoice, I summed the quantities of all individual items listed in that invoice. This provided a clear view of the overall volume of products involved in each transaction.

3. Is order UDF

```
# 3. Check if the transaction is an ORDER
def is_order(t_type):
    if t_type=='ORDER':
        return 1
    else:
        return 0
```

I used an if-else conditional statement to determine whether an invoice corresponds to a valid order.

4. Is return UDF

```
# 4. Check if the transaction is a RETURN
def is_return(t_type):
    if t_type=='RETURN':
        return 1
    else:
        return 0
```

I employed an if-else conditional structure to identify whether an invoice was associated with a return transaction.

Create Spark Session:

```
#Starting Spark Session
spark=SparkSession.builder.appName("spark-streaming").getOrCreate()
spark.sparkContext.setLogLevel("WARN")
```

A Spark session named "spark-streaming" is created.

Connect to Kafka:

```
#Fetching data from kafka
lines=spark.readStream\
    .format("kafka")\
    .option("kafka.bootstrap.servers","18.211.252.152:9092")\
    .option("startingOffsets","earliest")\
    .option("failOnDataLoss","false")\
    .option("subscribe","real-time-project")\
    .load()
```

It establishes a connection to a Kafka server and subscribes to the "real-time-project" topic.

Schema Definition:

```
#defining schema for incoming messages
jsonSchema = StructType() \
    .add("invoice_no", LongType()) \
    .add("country", StringType()) \
    .add("timestamp", TimestampType()) \
    .add("type", StringType()) \
    .add("items", ArrayType(StructType([
        StructField("SKU", StringType()),
        StructField("title", StringType()),
        StructField("unit_price", FloatType()),
        StructField("quantity", IntegerType())
    ])))
```

The schema for the incoming data is defined.

Convert Data Stream Messages:

```
#converting the json stream messages to Spark Dataframe
orders = lines.select(from_json(col("value").cast("string"),
                                jsonSchema).alias("data")).select("data.*")
```

The JSON-formatted messages from Kafka are parsed and converted into a Spark DataFrame for further processing.

Register UDF and Add New Column:

```
#Creating UDFs for applying on the dataframe
total_cost_udf = udf(calc_total_cost, DoubleType())
total_items_udf = udf(calc_total_items, IntegerType())
is_order_udf = udf(is_order, IntegerType())
is_return_udf = udf(is_return, IntegerType())
```

```
#apply UDFs on Spark Dataframe 'orders'
orders_df = orders \
    .withColumn("total_cost", total_cost_udf(orders.items, orders.type)) \
    .withColumn("total_items", total_items_udf(orders.items)) \
    .withColumn("is_order", is_order_udf(orders.type)) \
    .withColumn("is_return", is_return_udf(orders.type))
```

User-defined functions (UDFs) are utilized on the DataFrame to create additional columns: **"total_cost"**, **"total_items"**, **"is_order"**, and **"is_return"**.

Writing to console:

```
#writing the data to console
streaming_query = orders_df \
    .select("invoice_no", "country", "timestamp", "total_cost", "total_items", "is_order", "is_return") \
    .writeStream \
    .outputMode("append") \
    .format("console") \
    .option("truncate", "false") \
    .trigger(processingTime = "1 minute") \
    .start()
```

Aggregating KPI:

```
#aggregating by time
time_agg = orders_df \
    .withWatermark("timestamp", "1 minute") \
    .groupBy(window("timestamp", "1 minute", "1 minute")) \
    .agg(sum("total_cost").alias("total_sales_volume"),
        count("invoice_no").alias("OPM"),
        avg("is_return").alias("rate_of_return"),
        avg("total_cost").alias("average_transaction_size")) \
    .select("window", "OPM", "total_sales_volume", "average_transaction_size", "rate_of_return")
```

Two distinct aggregations are carried out

Writing to HDFS:

```
#writing the time-aggregated data to HDFS
agg_time_stream_query = time_agg.writeStream \
    .format('json') \
    .outputMode("append") \
    .option("truncate","false") \
    .option("path","/user/ec2-user/Timebased-KPI") \
    .trigger(processingTime="1 minute") \
    .start()
```

The aggregated key performance indicators (KPIs) are stored in HDFS as JSON files.

Aggregating KPI's by Time and Country:

```
#aggregating by time and country
time_country_agg = orders_df \
    .withWatermark("timestamp", "1 minute") \
    .groupBy(window("timestamp", "1 minute", "1 minute", "country")) \
    .agg(sum("total_cost").alias("total_sales_volume"),
        count("invoice_no").alias("OPM"),
        avg("is_return").alias("rate_of_return")) \
    .select("window", "country", "OPM", "total_sales_volume", "rate_of_return")
```

Writing to HDFS:

```
#writing time and country aggregated data to HDFS
agg_country_stream_query = time_country_agg.writeStream \
    .format("json") \
    .outputMode("append") \
    .option("truncate","false") \
    .option("path","/user/ec2-user/Country-and-timebased-KPI") \
    .trigger(processingTime="1 minute") \
    .start()
```

Termination:

```
#waiting for termination
streaming_query.awaitTermination()
agg_time_stream_query.awaitTermination()
agg_country_stream_query.awaitTermination()
```

